49. RAG Service in Go: Chunk → Embed → Retrieve → Generate

49. RAG Service in Go: Chunk → Embed → Retrieve → Generate

Phase: Phase 7 — AI engineering in Go
Estimated time: 2–3 days
Slug: `49-rag-service`

Learning objectives

- Chunk text with overlap
- Orchestrate embed+retrieve+generate
- Cite sources in answers
- Evaluate retrieval quality

Prerequisites

Complete prior numbered lessons. Type every example yourself.

RAG pipeline

1. **Chunk** documents (size + overlap)
2. **Embed** chunks → vectors
3. On query: embed query → **retrieve** top-K chunks
4. **Generate** answer with LLM using retrieved context
5. Return answer + citations

Go orchestration

Go shines as the reliable orchestrator: timeouts, concurrency, observability, auth.

```
chunks := Chunk(doc, 500, 50)
// embed + store
hits := store.TopK(ctx, queryVec, 5)
answer, err := llm.Generate(ctx, prompt(hits, question))
```

Keep prompts and models versioned. Measure hit rate and groundedness.

Common mistakes (especially from Python instincts)

- Huge chunks blowing context windows
- No citations
- Retrieving without metadata filters

Exercises

Work in `exercises/49-rag-service/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/49-rag-service/` until you have an honest failing attempt.

Self-check

1. Why overlap in chunking?
2. What are citations for?
3. Where does Go sit in RAG?

Further reading

- https://www.anthropic.com/engineering